

Project-2 of “Neural Network and Deep Learning”

Yanwei Fu

May 8, 2026

Abstract

(1) This is the second project of our course. The deadline is *23:59, June 14th, 2026*. Please upload the report via elearning.

(2) The goal of your write-up is to document the experiments you’ve done and your main findings. So be sure to explain the results. Hand in a single PDF file of your report. Enclose a Github link to your codes in your submitted file. You should also provide a link to your dataset and your trained model weights in your report. You may upload the dataset and model into Google Drive or other Netdisk service platform. Also put the name and Student ID in your paper. Lack of code link or model weights link will lead to a penalization of scores.

(3) About the deadline and penalty. In general, you should submit the paper according to the deadline of each project. The late submission is also acceptable; however, you will be penalized 10% of scores for each week’s delay.

(4) For junior students, we recommend to start with Mindspore tutorial on CIFAR 10 example:

https://mindspore.cn/docs/api/zh-CN/r1.5/api_python/dataset/mindspore.dataset.Cifar10Dataset.html

1 Train a Network on CIFAR-10 (60%)

CIFAR-10 [4] is a widely used dataset for visual recognition task. The CIFAR-10 dataset (Canadian Institute For Advanced Research) is a collection of images that are commonly used to train machine learning and computer vision algorithms. It is one of the most widely used datasets for machine learning research. The CIFAR-10 dataset contains 60,000 32×32 color images in 10 different classes. The 10 different classes represent airplanes, cars, birds, cats, deer, dogs, frogs, horses, ships, and trucks (as shown in Figure 1). There are 6,000 images of each class. Since the images in CIFAR-10 are low-resolution (32×32), this dataset can allow us to quickly try our models to see whether it works.

In this project, you will train neural network models on CIFAR-10 to optimize performance. Report the best test error you are able to achieve on this dataset, and report the structure you constructed to achieve this.

1.1 Getting Started

1. You may download the dataset from the official website [2] or use the torchvision package. Here is a demo provided by PyTorch [1].

```
1 import torch
2 import torchvision
3 import torchvision.transforms as transforms
```

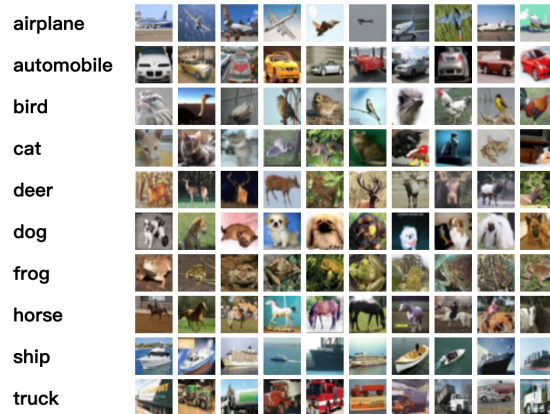


Figure 1: CIFAR-10 Dataset

```

4 transform = transforms.Compose( [transforms.ToTensor(), transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5,
0.5))] )
5 trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
6 trainloader = torch.utils.data.DataLoader(trainset, batch_size=4, shuffle=True, num_workers=2)
7 testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
8 testloader = torch.utils.data.DataLoader(testset, batch_size=4, shuffle=False, num_workers=2)

```

When setting *download=True* in line 5, it will download the dataset to the defined *root* path automatically. For the construction, training and testing the neural network model, you may read the tutorial [1] as a start.

2. Your network will have all the following components:(16%)
 - (a) Fully-Connected layer;
 - (b) 2D convolutional layer;
 - (c) 2D pooling layer;
 - (d) Activations;
3. Your network may have some of (at least one) the following components:(8%)
 - (a) Batch-Norm layer;
 - (b) Drop out;
 - (c) Residual Connection;
 - (d) Others;
4. To optimize your network, you will try all the following strategies:(8%)
 - (a) Try different number of neurons/filters;
 - (b) Try different loss functions (with different regularization);
 - (c) Try different activations;

5. To optimize your network, you may select one of the following strategies: (8%)
 - (a) Try different optimizers using *torch.optim*;
 - (b) Implement an optimizer for a network including 2(a)-(d) by yourself (You will **not** use things like *torch.nn*, *torch.sigmoid*, etc. You may use things like *torch.matmul*, etc) , and use *torch.optim* to optimize your full model;
 - (c) Implement an optimizer for your full model by yourself;
6. Reveal the insights of your network, say, visualization of filters, loss landscape, network interpretation. (8%)

1.2 Scores of this task

Yes, you are totally free to use any component in torch, pytorch or other deep learning toolbox. Then how do we score your project in this task? We care about

- (1) (12%) The classification performance of your network, in term of total parameters of network, network structure, and training speed. For the projects of similar results, we will check the total parameters, and network structure, or whether any new optimization algorithms are used to train the network.
- (2) Any insightful results and interesting visualization of the learned model, and training process, or anything like this.
- (3) You can report the results from more than one network; but if you directly utilize the public available models without anything changes, the scores of your projects would be slightly penalized.

2 Batch Normalization (30%)

Batch Normalization (BN) is a widely adopted technique that enables faster and more stable training of deep neural networks (DNNs). The tendency to improve accuracy and speed up training have established BN as a favorite technique in deep learning. At a high level, BN is a technique that aims to improve the training of neural networks by stabilizing the distributions of layer inputs. This is achieved by introducing additional network layers that control the first two moments (mean and variance) of these distributions.

In this project, you will first test the effectiveness of BN in the training process, and then explore how does BN help optimization. The sample codes are provided by Python.

2.1 The Batch Normalization Algorithm

Here we primarily consider BN for convolutional neural networks. Both the input and output of a BN layer are four dimensional tensors, which we refer to as $I_{b,c,x,y}$ and $O_{b,c,x,y}$, respectively. The dimensions corresponding to examples within a batch b , channel c , and two spatial dimensions x, y respectively. For input images the channels correspond to the RGB channels. BN applies the same normalization for all activations in a given channel,

$$O_{b,c,x,y} \leftarrow \gamma_c \frac{I_{b,c,x,y} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c \quad \forall b, c, x, y.$$

Here, BN subtracts the mean activation $\mu_c = \frac{1}{|\mathcal{B}|} \sum_{b,x,y} I_{b,c,x,y}$ from all input activations in channel c , where $\mathcal{B}$ contains all activations in channel c across all features b in the entire mini-batch and all spatial x, y , locations. Subsequently, BN divides the centered activation by the standard deviation σ_c (plus ϵ for numerical stability) which is calculated analogously. During testing, running averages of the mean and variances are used. Normalization is followed by a channel-wise affine transformation parametrized through γ_c, β_c , which are learned during training.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Figure 2: ConvNet configurations (shown in columns) of VGG model

Dataset. To investigate batch normalization we will use following experimental setup: image classification on CIFAR-10 with a network has the same architecture as VGG-A except the size of Linear layers is smaller since input assumed to be $32 \times 32 \times 3$, instead of $224 \times 224 \times 3$. And all sample codes are implemented based on *Pytorch*.

You can run *loaders.py* to download CIFAR-10 dataset, and output some examples to familiarize yourself with the data storage format. Note that if you are using a remote server, you need to use *matplotlib.pyplot.savefig()* function to save the plot results, and then download to local to view.

2.2 VGG-A with and without BN (15%)

In this section, you will compare the performance and characteristics of VGG-A with and without BN. We encourage you to extend and modify the provided code for clearer and more convincing experimental results. Note that you should understand the code first instead of using it as a black box.

If you want to use partial dataset to train for faster results, set *n_items* to meet your wish. The basic VGG-A network is implemented in *VGG.py*, you can train this network first to understand the overall network architecture and view the training results. Then write a class *VGG_BatchNorm* to add the BN layers to the original network, and finally visualize the training results of the two for comparison. Sample code for training and visualizing has been included in *VGG_Loss_Landscape.py*.

2.3 How does BN help optimization? (15%)

It is not enough to just use BN. We should understand why it can play a positive role in our optimization process so as to have a more comprehensive understanding of the optimization process of deep learning and select the

appropriate network structure according to the actual situation in the future works. You may want to check some papers, e.g., [3].

So what stands behind BN? After all, in order to understand how BN affects the training performance it would be logical to examine the effect that BN has on the corresponding optimization landscape. To this end, recall that our training is performed using gradient descent method and this method draws on the first-order optimization paradigm. In this paradigm, we use the local linear approximation of the loss around the current solution to identify the best update step to take. Consequently, the performance of these algorithms is largely determined by how predictive of the nearby loss landscape this local approximation is.

Recent research results show that *BN reparametrizes the underlying optimization problem to make its landscape significantly more smooth*. So along this line, we are going to measure:

1. Loss landscape or variation of the value of the loss;
2. Gradient predictiveness or the change of the loss gradient;
3. Maximum difference in gradient over the distance.

2.3.1 Loss Landscape

To test the impact of BN on the stability of the loss itself, *i.e.*, its Lipschitzness, for each given step in the training process, you should compute the gradient of the loss at that step and measure how the loss changes as we move in that direction. That is, at a particular training step, measure the variation in loss. You can do as following for a simple implementation:

1. Select a list of learning rates to represent different step sizes to train and save the model (i.e. [1e-3, 2e-3, 1e-4, 5e-4]);
2. Save the training loss of all models for each step;
3. Maintain two lists: *max_curve* and *min_curve*, select the maximum value of loss in all models *on the same* step, add it to *max_curve*, and the minimum value to *min_curve*;
4. Plot the results of the two lists, and use *matplotlib.pyplot.fill_between* method to fill the area between the two lines.

Use the same approach for VGG-A model with BN. Finally, try to visualize the results from VGG-A with BN and without BN on the same pic for more intuitive results.

For your better understanding, we provide sample code for visualizing the loss landscape (VGG_Loss_Landscape.py). You need to understand the code and train different models to reproduce the results of the Figure 2. Please feel free to modify and improve the sample code and report your choice of learning rates. Most importantly, show your final comparison results with the help of *matplotlib.pyplot*.

References

- [1] https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html.
- [2] <https://www.cs.toronto.edu/~kriz/cifar.html>.

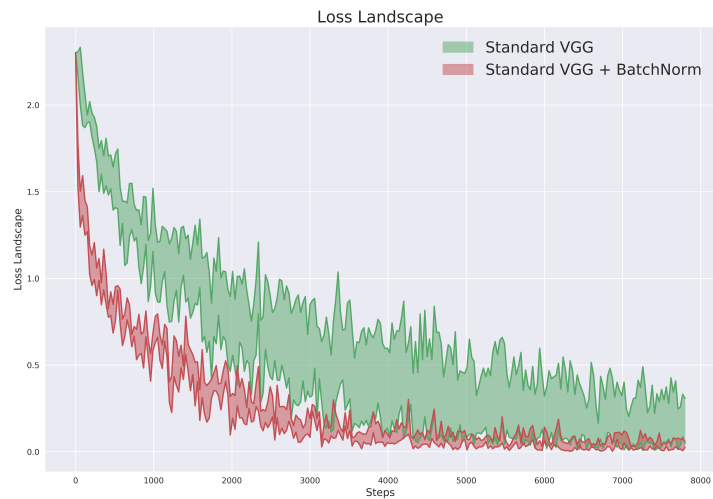


Figure 3: Loss landscape example

[3] How does batch normalization help optimization? In *NeurPIS*, 2018.

[4] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.